몬테카를로 방법 (Monte Carlo Methods)

모델 없이 배운다: MC 예측 · MC 제어와 블랙잭 실습 · 중요도 샘플링과 MC의 한계

Machine Learning 2 · 5주차

한국과학영재학교 (KSA)

Week 5

이번 주차 개요

Week 4의 동적계획법은 **전이확률** $P(s'|s,a)$ **를 정확히 안다**는 전제에 기대고 있었다. **몬테카를로 (MC)**는 그 모델을 통째로 버린다 — **에피소드를 끝까지 실제로 플레이하고, 결과(실제로 받은 리턴)를 그대로 가치의 추정치**로 쓴다.

이 주차를 마치면

- 리턴의 **평균**이 왜 가치함수로 수렴하는지 설명하고, 전이확률 단 한 줄도 쓰지 않은 채 **첫방문 MC 예측**으로 상태가치를 추정한다.
- 예측에서 **제어**로 $Q(s,a)$ 를 추정해 그리디 정책 개선으로 더 나은 정책을 만들고, ε-greedy 탐험이 왜 필요한지 설명한다.
- off-policy 에서 리턴을 그냥 평균내면 **편향**이 생기는 이유와 **중요도 샘플링** (확률의 비율 ρ)으로 보정하는 원리를 말한다.
- ρ 의 **분산이 에피소드 길이에 대해 지수적으로 폭발**하는 이유를 짚고, on/off-policy 트레이드오프를 정리한다.

세 차시

1. **5.1 모델 없이 배운다: MC 예측** – 잔디깎는 기 MDP 에서 DP 의 정확해와 첫방문 MC 가 **같은 숫자**에 도달하는 과정, 무편향성 증명, “마지막 방문” 함정.
2. **5.2 MC 제어와 블랙잭 실습** – $Q(s,a)$ 추정 + 그리디 개선, ε-greedy 로 **블랙잭 기본 전략표**를 재현.
3. **5.3 중요도 샘플링과 MC 의 한계** – off-policy 편향 보정, 분산 지수 폭발, continuing task 한계.

한 줄 감각: **모델 없이 배운다**(이 장), **기다림 없이 배운다**(Week 6 TD) — 강화학습 지도의 두 축이 하나씩 풀려나간다.

5.1 모델 없이 배운다: MC 예측

DP 의 전제와 MC 의 질문

DP 의 전제

- 벨만방정식을 **풀어야** 답에 도달.
- 전이확률 $P(s'|s,a)$ 가 **정확히** 필요.
- model-based 강화학습의 시작.

카드 게임 비유: 딜러의 "확률표"를 미리 손에 쥐고 있는 사람은 없다 — 그런데도 사람은 **게임을 반복해서 해보는 것만으로** 점점 좋은 전략을 배운다.

MC 의 질문

- 모델을 **전혀 몰라도** 배울 수 있는가?
- 에피소드를 끝까지 플레이하고, **실제로 받은 리턴**을 가치의 추정치로 쓴다.
- model-free 학습의 시작점.

여전히 남은 제한

리턴을 계산하려면 **에피소드가 끝날 때까지 기다려야 한다**. 이 기다림을 없애는 것이 Week 6 **시간차 학습 (TD)**.

DP vs MC — “모델의 유무”라는 축

	동적계획법 (DP)	몬테카를로 (MC)
전제	P 를 **안다고** 가정	P 를 **모른다고** 가정
방법	벨만방정식을 **푼다**	경험을 **평균낸다**
가치 갱신	(모델로 즉시)	**에피소드 종료 시**
접근	value-based (가치함수 중심)	value-based (가치함수 중심)

모델 유무와 가치 갱신 시점이 알고리즘을 나눈다

핵심 아이디어: 기댓값을 “평균”으로 치환한다

가치함수의 정의:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[\, G_t \mid s_t = s \,]$$

통계학의 기본 원리 (대수의 법칙)

기댓값은 **많은 샘플의 평균**으로도 근사할 수 있다.

정책 π 를 따라 에피소드를 여러 번 플레이하고, 상태 s 를 방문했을 때 이후 **실제로 받은 리턴**을 모아 평균 내면 그것이 곧 $V^{\pi}(s)$ 의 추정치.

기댓값을 평균으로 치환

왜 강화학습에서 “혁명적”이었는가

- Week 4 에서 벨만방정식을 “풀기” 위해 전이확률 P 가 필요했다면, MC 는 그 방정식을 **풀지 않는다**.
- 벨만 **동등성** — 방정식의 양변이 실제로 플레이할 때 같아야 한다는 사실 — 을 이용해 **플레이한 기록 자체가 답**이 되도록 한다.

정리

MC 예측은 “**기댓값 = 대수의 법칙에 의한 샘플 평균**”이라는 가장 기본적인 원리를 강화학습에 그대로 옮긴 것 — 전이확률이라는 “모델”을 전혀 쓰지 않고도 $V^{\pi}(s)$ 에 수렴한다.

역사: 카지노 도시에 이름을 건 알고리즘

- 이름의 기원: 모나코 공국의 카지노 도시 **몬테카를로**.
- 1940년대, MIT·맨해튼 프로젝트: Ulam, von Neumann, Metropolis (Ulam & Metropolis, 1949).
- 풀어야 할 문제: **중성자 이동** — 물질 안에서 중성자가 무작위로 여기저기 튕기는 과정.
- 이산 확률방정식을 정확히 풀어 계산하는 것은 불가능에 가까움.

해법

중성자가 어디로 가는지 **주사위처럼 던져서** 충분히 많이 추적해보자 — **무작위성을 도구**로 쓰는 계산법, **몬테카를로 방법**.

RL에서의 MC도 본질은 같음

전이확률을 몰라도, **무작위 전이를 실제로 겪어본 기록(에피소드)의 평균**이 기댓값을 대신한다. 카지노에서 주사위를 돌리는 것과 시뮬레이터에서 에피소드를 돌리는 것은 **같은 행위**.



몬테카를로 카지노 (모나코) — 이름의 기원

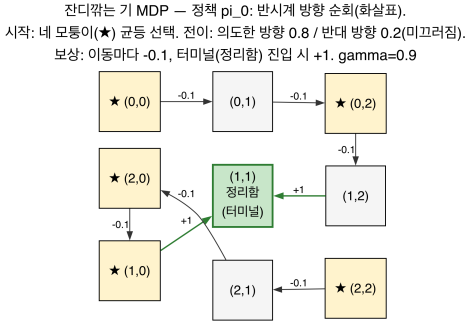


John von Neumann (1903–1957)
맨해튼 프로젝트, 몬테카를로 방법 공동 개발



Stanislaw Ulam (1909–1984) — Ulam & Metropolis, “The Monte Carlo Method” (1949)
중성자 이동 문제에서 무작위 표본으로 기댓값을 추정하는 계산법을 제안

예제 환경: 잔디깎는 기 (Lawnmower) MDP



3×3 격자, 정책 π_0: 반시계/시계 방향 순회 → 중앙 정리함.

- 3×3 격자. 네 모퉁이 $(0,0), (0,2), (2,0), (2,2)$ 가 **시작 상태**(매 에피소드 균등하게 1개 선택).
- 중앙 $(1,1)$ 이 **터미널**(정리함) — 들어가면 에피소드 끝.
- **전이**: 의도한 방향 0.8, 미끄러져 **정반대** 방향 0.2 (벽에 막히면 제자리).
- **보상**: 이동마다 -0.1, 터미널 진입 시 $+1$. $\gamma = 0.9$.

"모델이 있다"는 뜻은 0.8/0.2 를 **우리가 알고 있느냐**의 문제. MC 는 이 숫자를 몰라도 된다.

DP 로 풀면: 정확값 (정답 먼저 보고두기)

벨만방정식 반복 대입으로 정확값 구하기

상태	$V^{\pi_0}(s)$ (정확값)
$(0,0), (2,2)$ (모퉁이)	$+0.286$
$(0,1), (2,1)$ (모퉁이 옆)	$+0.465$
$(0,2), (2,0)$ (루프의 시작)	$+0.713$
$(1,2), (1,0)$ (터미널 바로 앞)	$+0.951$
$(1,1)$ (터미널)	0

첫방문 MC 예측: 알고리즘

1. 에피소드 π 로 샘플링.
2. 에피소드를 **뒤에서부터** 훑으며 리턴을 재귀 계산: $G_t = R_t + \gamma\, G_{t+1}$.
3. 각 상태의 **첫 방문** 시점의 G_t 로 $V(s)$ 를 갱신(증분 평균).

왜 뒤에서부터 훑나

리턴 $G_t = R_t + \gamma G_{t+1}$ 이 재귀적으로 정의되므로, 맨 마지막 스텝부터 거꾸로 누적하면 **한 번의 순회**로 매 스텝의 G_t 를 전부 계산 — Week 4 “터미널부터 거꾸로 대입” 패턴과 같은 아이디어.

첫방문 MC 예측: 코드

```python
def mc_prediction(policy, env_sample_episode, n_episodes, gamma):
    # env_sample_episode(policy) -> [(state, action, reward), ...]
    returns_sum = {}
    returns_count = {}
    for _ in range(n_episodes):
        episode = env_sample_episode(policy)
        G = 0.0
        visited = set()
        for t in reversed(range(len(episode))):    # 뒤에서부터: G_t 재귀
            s, a, r = episode[t]
            G = r + gamma * G
            if s not in visited:                   # 첫방문만카운트
                visited.add(s)
                returns_sum[s] = returns_sum.get(s, 0.0) + G
                returns_count[s] = returns_count.get(s, 0) + 1
    return {s: returns_sum[s] / returns_count[s] for s in returns_sum}
```

손으로: 한 에피소드의 리턴 계산 (미끄러짐 없음)

스텝	위치	이동	보상 R	리턴 G
0	$(0,0)$	$\rightarrow (0,1)$	-0.1	$-0.1 + 0.9 \times 0.620 = +0.458$
1	$(0,1)$	$\rightarrow (0,2)$	-0.1	$-0.1 + 0.9 \times 0.800 = +0.620$
2	$(0,2)$	$\rightarrow (1,2)$	-0.1	$-0.1 + 0.9 \times 1.000 = +0.800$
3	$(1,2)$	$\rightarrow (1,1)$ 터미널	$+1.0$	$+1.0$

리턴은 터미널 스텝부터 거꾸로

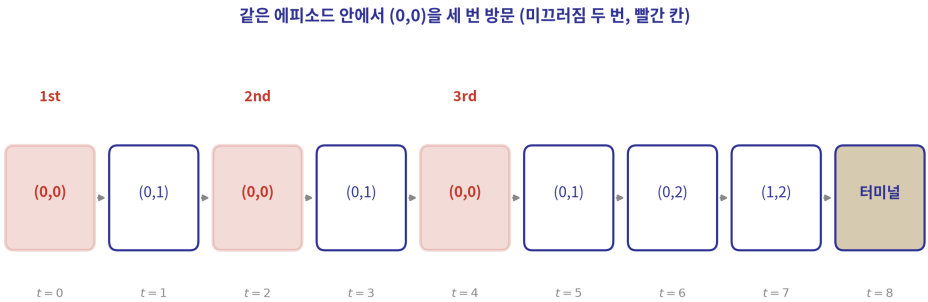
상태마다 다른 리턴이 방문 추정치에 들어간다

첫방문이 실제로 “한번”이 아님을 보는 예

스텝	위치	보상 R	리턴 G
0	$(0,0)$	-0.1	-0.139
1	$(0,1) \to$ (미끄러짐) $(0,0)$	-0.1	-0.043
2	$(0,0)$ (두 번째 방문)	-0.1	$+0.063$
3	$(0,1)$	-0.1	$+0.181$
4	$(0,0)$ (세 번째, 다시 미끄러짐)	-0.1	$+0.312$
5∼8	(정상 루프 $\to$ 터미널)	$\cdots$	$\to +1.0$

첫방문 MC는 첫 방문 리턴만 사용

그림으로: 한 에피소드 안에서의 재방문



왼쪽 표(p13)의 경로를 시간순으로 펼친 것 — $(0,0)$ 이 세 번(미끄러짐 두 번 포함) 등장하지만, 첫방문 MC는 **1st 방문의 리턴만** 카운트하고 2nd·3rd 방문은 버린다.

MC 가 DP 와 만나는 순간: 20만 에피소드

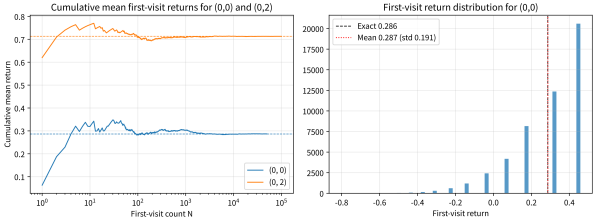
상태	정확값 V^{π_0}	첫방문 MC $\hat{V}$	오차	방문 횟수
$(0,0)$	$+0.286$	$+0.288$	$+0.002$	50,546
$(0,1)$	$+0.465$	$+0.466$	$+0.001$	50,546
$(0,2)$	$+0.713$	$+0.713$	-0.000	100,155
$(1,2)$	$+0.951$	$+0.951$	-0.000	100,155
$(2,0)$	$+0.713$	$+0.713$	-0.000	99,845
$(2,2)$	$+0.286$	$+0.286$	-0.001	49,967

전이가 0.8/0.2 라는 **숫자를 코드 어디에도 쓰지 않았는데도**, 8개 비터미널 상태 전부가 정확값과 0.002 이내에 붙어 있다.

핵심 결과

DP 는 “방정식을 풀어서”, MC 는 “경험을 평균내어서” 같은 답에 도달했다.

수렴의 “느낌” — 리턴이 흔들리고 평균이 진정된다



좌: $(0,0)$, $(0,2)$ 의 첫방문 리턴 누적 평균(로그 스케일). 우: $(0,0)$ 의 첫방문 리턴 분포.

왼쪽 그래프를 천천히 읽으면 MC 학습의 느낌이 잡힌다:

- $N = 100$: $(0,0)$ 누적 평균 0.262 (정확값 0.286 에서 벗어나 있음)
- $N = 1000$: 0.295
- $N = 10000$: 0.288
- $N = 100000$: 0.287 **진정(수렴)**

오른쪽은 “왜 처음엔 흔들리는가”: $(0,0)$ 의 첫방문 리턴 분포가 0∼1 로 넓게 퍼져 있음(표준편차 ≈ 0.19, 평균 ≈ 0.285).

왜 처음엔 흔들리는가 — $\sigma/\sqrt{N}$

- **단일 에피소드의 리턴은 잡음이 큰 단일 관측치**. 평균의 표준오차는

$$\frac{\sigma}{\sqrt{N}}$$

으로 줄어든다 — $N = 10{,}000$ 이면 오차 범위 $\approx \pm 0.002$ 로 좁혀진다.

핵심 정리 (5.1)

MC 예측은 전이확률이라는 "모델"을 전혀 쓰지 않고도 에피소드를 충분히 많이 돌리면 $V^{\pi}(s)$ 에 수렴한다. 대가는 (1) 에피소드가 **끝날 때까지** 결과를 모른 채 기다려야 하고, (2) 리턴의 **분산**이 크므로 같은 정확도에 더 많은 에피소드가 필요하다 — (1)은 Week 6 TD, (2)는 5.3 중요도 샘플링에서 다시 문제된다.

왜 첫방문 MC 는 무편향인가 — 두 줄의 직관

MC 예측의 무편향성

$$\mathbb{E}_\pi[\, G_t \mid s_t = s,\ \text{s가 t 이전에 안 나온} \,] \;=\; V^\pi(s)$$

1. **마르코프 성질** — “이전에 s 가 안 나왔다”는 조건이 붙어도 미래는 **현재 상태만으로 결정**되므로 값이 바뀌지 않는다.
2. **대수의 법칙** — 각 에피소드의 “첫 방문 리턴”은 같은 분포(기댓값 $V^\pi(s)$)에서 나온 **독립 샘플**. 독립 샘플의 평균은 그 분포의 기댓값으로 수렴.

자주 하는 실수: 뒤에서부터 훑으면 “마지막 방문”

```
for t in reversed(range(len(episode))):    # 뒤에서부터
    s, a, r = episode[t]
    G = r + gamma * G
    if s not in visited:    # 아직” 못봤다 ” = 이에피소드에서
                            # 가장늦게마지막으로 () 나온방문
        ...
```

reversed 순서에서 “아직 visited 에 없는 상태”를 만나면, 그 방문은 **그 상태에서 가장 늦게(마지막으로) 방문한 것** — 앞(시간 순서)에서부터 훑어야 첫방문을 센다.

상태	정확값	첫방문 MC	**마지막 방문 MC**
$(0,0)$	$+0.286$	$+0.288$	$+0.388$
$(0,1)$	$+0.465$	$+0.466$	$+0.542$
$(0,2)$	$+0.713$	$+0.713$	$+0.756$
$(1,2)$	$+0.951$	$+0.951$	$+1.000$

마지막 방문 MC 가 왜 높게 추정되는가

마지막 방문 MC는 정확값보다 높다

$$\mathbb{E}[\, G_t \mid s_t = s, \text{ 그 이후 } s \text{ 를 다시 만남}\,]$$

(수식으로: $\mathbb{E}[G \mid \text{last visit}] = 0.756$ 이 무조건 기대 $V^{\pi}(0,2) = 0.713$ 과 **0.043 차이** — 위 표의 $+0.756$ 과 정확히 일치.)

결론

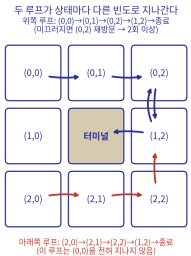
"뒤에서부터 훑는" 코드는 **편향된(biased) 추정치**를 만든다 — 에피소드를 어마어마하게 늘려도 0.756 근처에 머물고 0.713 에는 **영원히** 도달하지 못한다. **코딩 실수가 통계적 편향으로 바뀌는 대표적 예.**

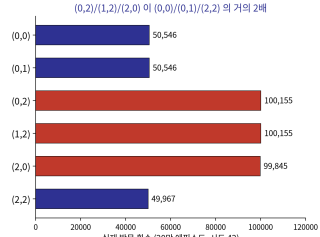
두 번째 실수: “평균 에피소드 길이”로 방문 횟수 추정하기

에피소드당 평균 길이로 방문 횟수 추정하기

원칙: 방문 횟수 = 신뢰도

(5.2절 블랙잭에서 보았던 그 원칙) 이 “실제 횟수”는 **계산해봐야** 알 수 있다. “평균 길이 $\times$ 에피소드 수”로 **대충 세는** 것은 방문이 **불균형한** 환경(루프가 여러 개인 잔디깎는 기, 낮은 합이 드문 블랙잭)에서 신뢰도를 **잘못 판단**하게 만든다.





위쪽 루프는 $(0, 2)$를, 아래쪽 루프는 $(0, 0)$를 건너뛴다 — 실제 방문 횟수(우)가 거의 2배 차이

자주 묻는 질문 (5.1) — MC vs DP

- **MC 예측은 Week 4 정책평가와 결국 같은 것을 계산하나요?** — 목표($V^\pi(s)$)는 같지만 **방법**이 다르다. 정책평가는 P 를 안다는 전제로 벨만방정식을 반복 대입(모델 기반), MC 예측은 P 를 전혀 모른 채 관찰된 리턴의 **평균**으로 추정(모델 프리) — 대수의 법칙에 의해 에피소드가 많아질수록 진짜 $V^\pi(s)$ 에 가까워진다.
- **에피소드가 짧으면 추정이 더 정확한가요?** — 아니다. 정확도를 높이는 건 그 상태를 **방문한 횟수** (더 많은 에피소드)이지 한 에피소드의 길이가 아니다. 다만 매우 긴 에피소드는 G_t 분산이 커져 같은 정확도에 더 많은 에피소드가 필요할 수 있다.

MC 가 "모델을 모른다"는 것이 DP보다 항상 이득인가요? — 아님. 모델이 정확하고 작다면(이 잔디깎는 기처럼) DP 가 **훨씬** 빠르고 정확하다 (20만 에피소드 + 오차 0.002 vs 8개 연립방정식을 **정확히** 풀기). MC 의 진짜 무대: "상태가 수천 수만 개, 전이를 사람이 손으로 쓸 수 없는" 문제 — 이 판단은 Week 15(모델 기반 RL과 Dyna)에서 다시 만난다.

자주 묻는 질문 (5.1) — γ 는 MC 에 영향을 주나

주겠다 — 단, **리턴의 정의**를 통해서만.

- $\gamma = 0.9$ 일 때 $(0, 0)$ 의 리턴은 $-0.1 + 0.9 \times G_{t+1}$ 처럼 미래 보상을 10%씩 할인해 계산.
- $\gamma = 1$ 이면 할인 없이 모든 보상을 통째로 더 — 에피소드가 **유한하고 끝이 나는** 과업(잔디깎는 기 처럼)이라면 $\gamma = 1$ 도 괜찮다. (5.2 블랙잭에서 $\gamma = 1$ 을 쓴 이유와 같은 맥락.)

정리

γ 는 "정답이 무엇인가"를 바꾸고, MC 는 그 "바뀐 정답"을 평균으로 재추정한다 — γ 를 바꾸면 **DP 의 정확값과 MC 의 수렴 목표가 동시에 바뀐다**.

5.2 MC 제어와 블랙잭 실습

예측에서 제어: V 대신 Q 를

V 대신 Q 를 추정

탐험을 어떻게 보장할 것인가

순수 탐욕적으로만 행동하면 **한 번도 시도하지 않은** (상태, 행동) 쌍은 영원히 가치를 알 수 없다 (Week 2.1 “탐욕적 선택의 함정”이 재발).

두 가지 탐험 전략: exploring starts vs ε-greedy

(1) 탐험적 시작 (exploring starts)

- 매 에피소드를 **무작위로 고른** (상태, 행동) 쌍에서 **강제로 시작**.
- 모든 (상태, 행동) 쌍이 결국 충분히 방문됨.
- 이론적으로 깔끔하지만, 임의의 상태·행동에서 "시작"할 수 있어야 하므로 **실전 환경에서는 항상 가능하지 않다**.

(2) ε-greedy (Week 2.1 의 그 전략)

- 매 스텝 **작은 확률로 무작위 행동**을 섞음.
- 시작 상태와 무관하게 모든 (상태, 행동) 쌍을 계속 조금씩 방문.
- **이번 실습은 ε-greedy.**

MC 제어: 코드 (첫방문 기준 Q 갱신)

```
def mc_control(env_step, n_states, n_actions, n_episodes,
               epsilon, gamma, start_state):
    Q = [[0.0] * n_actions for _ in range(n_states)]
    counts = [[0] * n_actions for _ in range(n_states)]

    def generate_episode():
        s, episode = start_state, []
        for _ in range(100):
            a = random.randrange(n_actions) if random.random() < epsilon \
                else max(range(n_actions), key=lambda x: Q[s][x])
            ns, r, done = env_step(s, a)
            episode.append((s, a, r))
            s = ns
            if done:
                break
        return episode

    for _ in range(n_episodes):
        episode = generate_episode()
        G, visited = 0.0, set()
        for t in reversed(range(len(episode))):
            s, a, r = episode[t]
            G = r + gamma * G
            if (s, a) not in visited:          # 상태(, 행동) 쌍방문
                visited.add((s, a))
                counts[s][a] += 1
                Q[s][a] += (G - Q[s][a]) / counts[s][a]  # 증분평균
    return Q
```

$Q[s][a]$ **갱신,** (s, a) **방문 기준**

MC 제어로 학습하는 것의 정체: Q^{π}

위 코드가 실제로 학습하는 것은 “최적 정책의 가치”가 아니라, **지금 ε-greedy 로 행동하는 그 정책 자체의 가치** $Q^{\pi}(s,a)$.

최종 정책은 탐욕적 정책으로 추출

on-policy 라는 뜻: 행동 정책 = 목표 정책

핵심 정리 (5.2)

ε-greedy MC 제어는 (1) 탐험을 섞은 **행동 정책** π 의 Q^{π} 를 추정하고, (2) 거기서 arg max 를 취해 **결정론적 정책**을 만들어낸다. “추정 대상”과 “최종 배포 정책”이 살짝 달라지는 이 미묘함은 Week 9 DQN 과 Week 11 PPO 에서도 계속 따라다닌다.

on-policy: 행동 정책 = 목표 정책

실습: 블랙잭에서 MC 제어로 학습하기

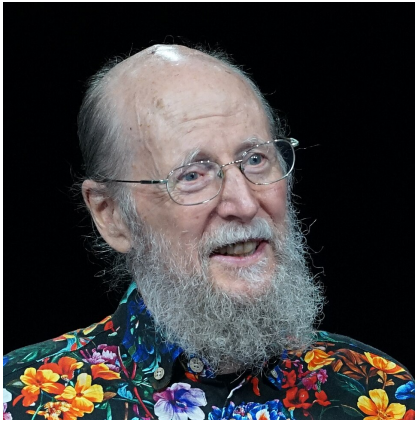
Sutton & Barto 의 RL 교재에서 MC 제어를 소개할 때 쓰는 **표준 예제**가 블랙잭이다.

- **상태**
  $s = (\text{플레이어 합}, \text{딜러의 보이는 카드})$
- **행동**: 0 = **스틱**(멈추기), 1 = **히트**(카드 더 받기)
- **보상**: $+1$(이김), -1(지면/버스트), 0(무)
- 스틱하면 **딜러가 17 이상**이 될 때까지 받는 과정을 한꺼번에 시뮬레이션.

왜 블랙잭인가

MC 제어의 두 가설(모델 불필요 + 에피소드가 있어야 함)을 **동시에** 가장 선명하게 보여준다.

- 카드 덱의 전이확률은 손으로 계산하기 번거롭지만 게임을 "돌려보면" 알 수 있다.
- 한 판은 **분명히 끝난다** (버스트 또는 딜러 마무리) — MC 의 "에피소드 끝까지 기다려 리턴 계산"이 그대로 적용.



Richard Sutton — Barto와 2024 ACM Turing Award
(강화학습 이론적 기초, Reinforcement Learning: An Introduction 공저)



실제 카지노 블랙잭 테이블 — 이번 실습이 재현하려는 게임

블랙잭: 에피소드 생성 코드

```
def gen_episode(Q, epsilon):
    player_sum = draw_card() + draw_card()
    dealer_showing = draw_card()
    s = (player_sum, dealer_showing)
    episode = []
    while True:
        if s[0] < 12:
            a = 1  # 12 미만이면무조건히트자명한    ( 선택)
        else:
            a = random.randrange(2) if (s not in Q or random.random() < epsilon) \
                else max(range(2), key=lambda x: Q[s][x])  # 스틱0=, 히트1=
        ns, r, done = step(s, a)
        episode.append((s, a, r))
        if done:
            break
        s = ns
    return episode
```

할인율 $\gamma = 1$ 을 쓰는 이유

- 블랙잭의 한 판은 **결정적으로 끝나는 유한 호라이즌** — 누군가 버스트하거나 딜러가 17 이상으로 마무리되는 순간 에피소드가 끝나므로, 보상이 "무한히 먼 미래"까지 이어지지 않는다.
- 그런 **유한하고 끝이 나는 과업**에서는 $\gamma = 1$ (보상을 통째로 가중하지 않는 것)이 자연스러운 선택.

보상을 $-1, 0, +1$ 로 두는 이유

"기대 수익이 0 근처의 게임"이라는 직관이 Q 값의 **부호와 크기**에 그대로 담기게 한다.
$Q(s, a) > 0$ 은 기대적으로 **이길** 확률이 더 크다, $Q(s, a) < 0$ 은 **지실** 확률이 더 크다.

결과: 카드 덱 확률분포를 계산하지 않은 전략

상태 (합, 딜러)	Q(스틱)	Q(히트)	선택
(20, 3)	$+0.633$	-0.839	스틱
(20, 10)	$+0.520$	-0.811	스틱
(12, 10)	-0.539	-0.253	히트

- **20이면 스틱이 압도적으로 높음**
- **12 + 딜러 10이면 히트가 유리**

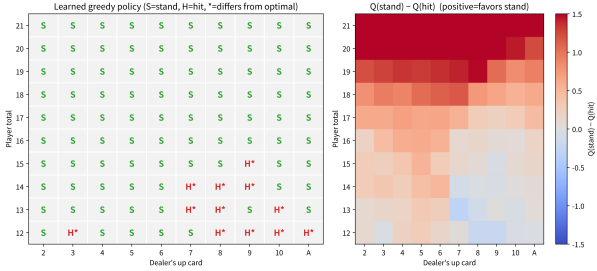
경험만으로 패턴 학습

학습한 전략을 "정답"과 비교하자

상태 (합, 딜러)	학습 Q(스틱)	학습 Q(히트)	정확 Q^*(스틱)	정확 Q^*(히트)	최적	방문(
(20, 3)	$+0.633$	-0.839	$+0.656$	-0.828	스틱	2
(20, 10)	$+0.520$	-0.811	$+0.540$	-0.832	스틱	3
(12, 10)	-0.539	-0.253	-0.532	-0.206	히트	2
(12, 1)	-0.366	-0.131	-0.380	-0.122	히트	2
(15, 10)	-0.550	-0.404	-0.532	-0.388	히트	3
(15, 2)	-0.391	-0.356	-0.363	-0.321	히트	3

여섯 상태 모두 **최적 행동이 일치**, 학습 Q 도 정확 값과 대체로 0.03 안팎. 특히 (15, 2)는 스틱(-0.39)과 히트 (-0.36)의 차이가 0.035에 불과한 **경계 상태**인데도 MC 가 올바른 쪽(히트)을 골랐다. 전체 110개 학습 상태로 넓히면 그리디 정책이 정확 최적 정책과 **104/110** (약 95%) 일치.

학습만으로 “기본 전략표”가 그려진다



좌: 학습된 전략(S=스틱/초록, H=히트/빨강, 별표=DP 와 다름). 우: Q(스틱)$-Q$(히트) 차이.

학습 정책을 “플레이어 합 × 딜러 오픈카드” 격자로 펼치면 **교과서적인 블랙잭 기본 전략표**가 거의 그대로 나타난다:

- 합 17 이상: 딜러 카드 무관 **모두 스틱**.
- 합 12∼13: **모두 히트**.
- 그 사이 14∼16: **딜러 카드에 따라 갈림** — 딜러가 강한 카드(10)를 보여줄수록 히트 쪽, 약한 카드(2∼6)를 보여줄수록 스틱 쪽.

핵심

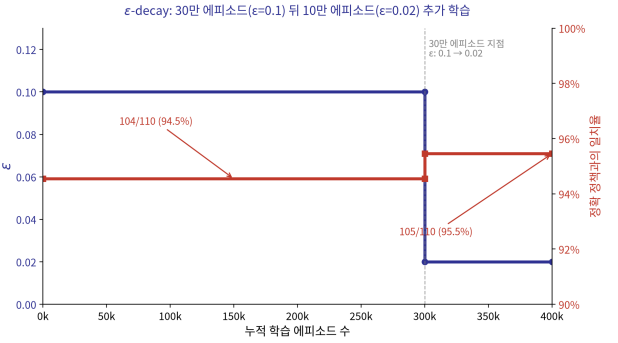
전이를 계산한 줄이 한 줄도 없는데, 시뮬레이션 40만 판만으로 이 표가 “그려진” 것 — 이것이 MC 제어의 핵심.

ε 를 줄여가며 학습: 탐험에서 착취로

ε-decay: 탐험에서 착취로

이 실습: 30만 에피소드($\varepsilon = 0.1$) 뒤에 10만 에피소드를 $\varepsilon = 0.02$ 로 추가 학습.
효과(작지만 실제로 측정됨):

- 정확 정책과의 일치율: **104/110** → **105/110**.
- $|Q - Q^*|$ 의 평균: 0.029 → 0.027.



ε 는 "상한선"인가, "속도"인가

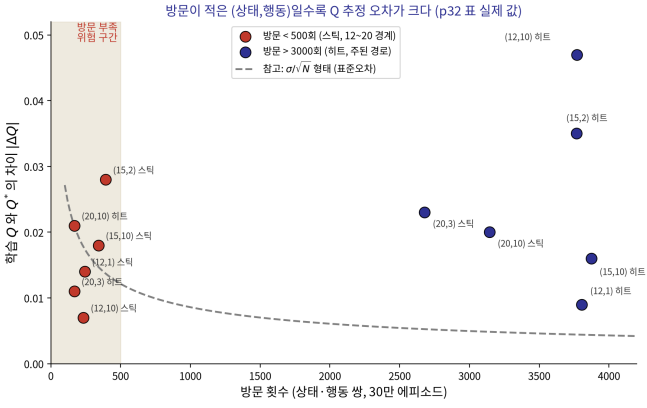
$\varepsilon > 0$ **인 한 행동 정책은 무작위 행동을 섞으므로, 그 정책의 가치**

ε 를 0으로 완전히 줄이면 어떨까

그때는 더 이상 새 (상태, 행동) 쌍을 방문하지 못해, 아직 제대로 추정되지 않은 상태의 Q 가 **영원히** 그 자리에서 멈춘다 — 탐험의 대가(낮은 즉석 보상)를 치르지 않으면 새로운 지식을 얻을 기회도 사라진다. **"탐험 vs 착취" 트레이드오프의 가장 정직한 모습.**

자주 하는 실수: 방문 횟수가 적은 상태의 Q 를 그대로 믿기

- MC 추정치는 “그 상태를 **몇 번 방문했는가**”에 직접적으로 의존.
- 방문 횟수가 적은 (상태, 행동) 쌍의 Q 값은 곧이곧대로 신뢰하기보다 **“더 많은 에피소드가 필요하다”는 신호**로 받아들이는 것이 안전.



두 번째 실수: 학습된 Q^{π} 를 곧 Q^{*} 처럼 읽기

학습된 Q는 Q*보다 낮게 측정된다

"그 ε-greedy 정책 아래에서 그 상태·행동의

평균 리턴이 약 -0.5 다"라고 읽어야 한다.

최적 정책의 "진짜" 가치를 알고 싶다면

5.3절의 **off-policy (중요도 샘플링)**이나 Week 6 의 **Q-learning** 이 필요하다.

자주 묻는 질문 (5.2) — MC 예측 vs MC 제어

코드적으로 얼마나 다른가요? — 거의 같다, 두 줄만 바뀐다.

MC 예측 (5.1)

- $V[s]$ 를 갱신.
- 상태 s 만 "방문"으로 셈.

MC 제어 (5.2)

- $Q[s][a]$ 를 갱신.
- **(상태, 행동) 쌍** (s, a) 를 방문 기준으로 삼음.

나머지는 **동일** (에피소드를 뒤에서부터 훑어 G_t 를 재귀 계산, 증분 평균으로 갱신).

이 "거의 같다"가 MC 의 미덕 — **예측을 할 줄 안다면 제어는 자동으로 따라온다.**

왜 V 만으로는 안 되고 Q 여야 하는가? — $V(s)$ 만으로는 "다음에 **어느 행동**을 골랐는지"가 사라져 다음 행동을 결정할 수 없다. $Q(s, a)$ 는 "지금 행동까지 포함"한 가치로 그 부족함을 메운다.

자주 묻는 질문 (5.2) — 실제 블랙잭에 쓸 수 있나, Q 부호 해석

- **학습한 정책을 실제 블랙잭에 쓰면 이길 수 있나요?** — 이 절의 단순화 모델(소프트 에이스 무시, 무한 카드 덱)로 학습한 정책은 **기본 전략의 골격**은 재현하지만, 실제 카지노 규칙(소프트 17, 멀티덱, 히트 온 17 등)에는 정확히 맞지 않는다. MC 의 강점은 "규칙이 아무리 복잡해도(전이확률이 계산 불가능해도) 시뮬레이션만 하면 전략을 배운다"는 것이지, "학습한 정책이 무조건 현역 프로보다 낫다"는 것이 아니다.
- Q **값의 부호(+/−)는 어떻게 해석하나?** — 블랙잭은 (공정한 게임이므로) 전체 기대 수익이 0 근처 이므로, 모든 상태·행동의 Q 가 동시에 양수/음수일 수 없고 **상태마다 양/음이 섞여 있어야 한다**. 학습된 Q 에서 (20, 3) 스틱 $+0.63$, (12, 10) 스틱 -0.54 로 부호가 반대인 것이 바로 이 "게임 전체는 0 근처" 구조를 반영한다.

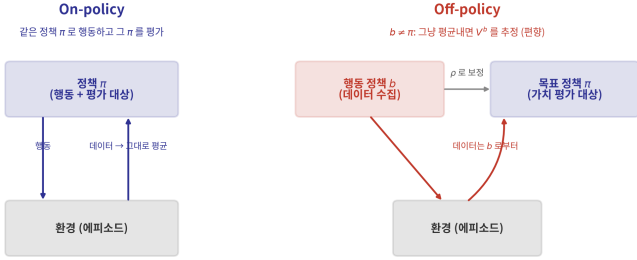
5.3 중요도 샘플링과 MC 의 한계

off-policy 라는 문제: 그냥 평균내면 편향

off-policy에서 그냥 평균내면 안 된다

행동 정책 b (behavior)
데이터를 **모을 때** 쓴 정책.

목표 정책 π (target)
가치를 **평가하려는** 정책.



$b \neq \pi$ 면 관찰된 리턴을 **그대로** 평균내는 것은 **편향된(biased) 추정**이다.

중요도 샘플링: 확률의 비율 ρ 로 보정

중요도 샘플링 은 각 에피소드의 리턴에 "이 궤적이 π 에서 나왔을 확률 대비 b 에서 나왔을 확률의 **비율**"을 가중치로 곱해 편향을 보정:

$$\rho \;=\; \prod_{t} \frac{\pi(a_t|s_t)}{b(a_t|s_t)}$$

왜 "확률의 비율"인가 — 평가하려는 값은 $V^{\pi}(s_0) = \mathbb{E}_{\pi}[G_0]$ 인데, b 에서 샘플을 모으려면 π 에 대한 기댓값을 b 에 대한 기댓값으로 바꿔야 한다 — 확률론의 기본 정리를 쓰는 것뿐:

$$\mathbb{E}_{\pi}[f] = \mathbb{E}_{b}\left[f \cdot \frac{p_{\pi}}{p_{b}}\right]$$

비율 2가 L 스텝 뒤 2^{L}로 분산 폭증

직관

목표 정책 π 라면 거의 선택하지 않았을 행동을 행동 정책 b 가 우연히 선택해서 얻은 궤적은, 그 **희소함**(ρ 가 작음)만큼 가중치를 **낮춰서** 반영.

- 가중치가 **에피소드 전체의 비율**이고 단 한 스텝의 비율이 아닌가 — 마르코프 성질 덕분에 스텝별 비율의 **곱**으로 분해.
- ρ 는 0 또는 1이 아니라 **4나 1/8처럼 "10배"**가 될 수 있다.

손으로 하는 중요도 샘플링: 5칸 회랑

5칸 GridWorld 설정

행동 정책 b 매 스텝 왼쪽/오른쪽을 **50-50** 무작위.

목표 정책 π **항상 오른쪽**(탐욕적).

두 정책의 가치를 **정확히** 알 수 있다(Week 4 의 역순 대입):

$$V^{\pi} = [0.729,\ 0.810,\ 0.900,\ 1.000,\ 0]$$

$$Q^{\pi}(2,\ \text{right}) = 0.900 > Q^{\pi}(2,\ \text{left}) = 0.729$$

즉 “칸 2에서 오른쪽이 더 좋은 정책”이 $V^{\pi}(2) = 0.9$ 다.

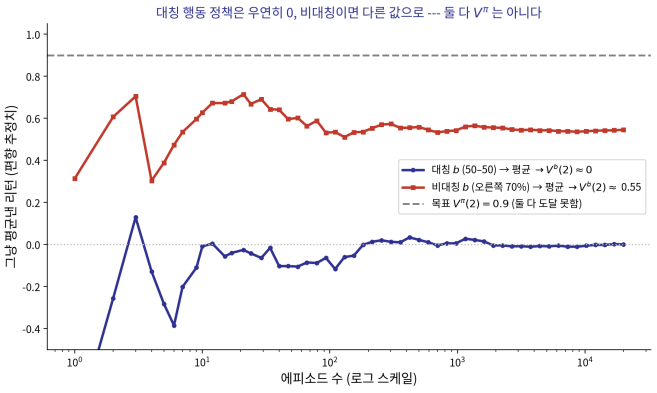
핵심 질문: b **의 데이터만 모아서** $V^{\pi}(2) = 0.9$ **를 추정할 수 있는가?**

편향의 함정: “그냥 평균”이 추정하는 것

평균은 $V^b(2)$ 추정

편향 (bias)

에피소드를 **아무리 많이 돌려도 해소되지 않는다**. (실습 노트북에서 2만 에피소드로 직접 확인: 그냥 평균은 $0.004 \to 0$ 에 수렴, 0.9에는 **절대 안 간다**.)



중요도 보정: 스텝별 비율

$$\frac{\pi(a_t|s_t)}{b(a_t|s_t)} = \begin{cases} 2 & a_t = \text{right} \\ 0 & a_t = \text{left} \end{cases}$$

다시 말해, **왼쪽이 한 번이라도 나오면 그 에피소드의** $\rho = 0$ — "π 라면 절대 이렇게 행동하지 않았을 궤적"이라 평가에서 **완전히 제외**. 오른쪽만 나오면(시작 2에서 오른쪽만 L번 $\rightarrow$ 종료 4) $\rho = 2^L$ 다.

에피소드 (시작 2)	리턴 G_0	ρ	$\rho \cdot G_0$
(2,오) $\rightarrow$ (3,오, 종료)	$0 + 0.9 \times 1 = 0.9$	$2 \times 2 = 4$	3.6
(2,좌) $\rightarrow$ (1,오) $\rightarrow$ (2,오) $\rightarrow$ (3,오, 종료)	$0.9^3 = 0.729$	0 (첫 스텝에 왼쪽)	0
(2,좌) $\rightarrow$ (1,$\cdots$) $\rightarrow$ (0, 종료)	≈ -1	0	0

ρG_0**는** $\{3.6, 0\}$ **두 값만 취한다**

그래도 정확히 맞는다 (편향 없음) — 대가는 분산

$$\mathbb{E}[\,\rho G_0\,] = 0.25 \times 3.6 = 0.9 = V^{\pi}(2)$$

정확히 맞는다 (편향 없음).

$$\mathsf{Var}[\,\rho G_0\,] = 0.25 \times 3.6^2 - 0.9^2 = 2.43$$

에피소드 길이 L일 때 확률 비율 2^L 배

$$\mathbb{E}_b[\rho] = 1, \qquad \mathsf{Var}_b[\rho] = 2^L - 1$$

$\mathbf{Var}[\rho] = 2^L - 1$: 지수 폭발

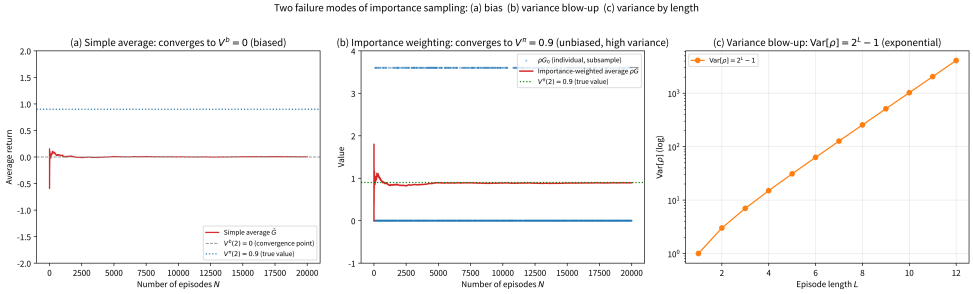
$\mathbf{Var}[\rho] = 2^L - 1$ 지수함수적 증가

L	$\mathrm{Var}[\rho] = 2^L - 1$
$L = 2$	3
$L = 4$	15
$L = 6$	63
$L = 8$	255
$L = 10$	1023

본질

편향은 없애주되 대가로 분산을 극단적으로 키운다 — 이것이 중요도 샘플링의 본질.

중요도 샘플링의 두 가지 실패 모드



좌: 그냥 평균(b 데이터)은 편향되어 $V^b = 0$ 에 수렴. 중앙: ρ 가중 평균은 편향 없이 $V^\pi = 0.9$ 에 수렴하나 점의 분산이 매우 큼. 우: ρ 분포가 에피소드 길이 L 에 따라 지수적으로 퍼짐($\mathrm{Var}[\rho] = 2^L - 1$).

“정확한 수렴” $\neq$ 실용적 — 에피소드 수가 지수적으로 폭증

길이 L 인 에피소드에서 ± 0.05 정확도를 원하면 필요한 에피소드 수는 대략

$$N \approx \left(\frac{2}{\delta}\right)^2 \cdot 2^L \cdot \mathbb{E}[G^2]$$

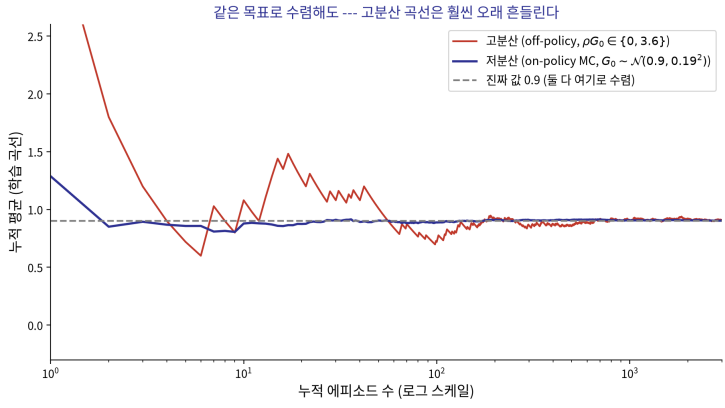
L 이 1 커질 때마다 **2배**. $L = 5$ 이면 10^4 급, $L = 20$ 이면 10^9 급 — **15 스텝이 길어진 대가로 필요한 데이터가 약 $2^{15} \approx 3$만 배**.

off-policy 학습 전반의 근본 난제

에피소드가 조금만 길어져도 **“수학적으로는 가능하지만 실제로는 불가능”**이 된다.

“분산이 크다”가 실전에서 무엇을 의미하나

- 학습 곡선이 안 정해진다
- 한두 에피소드가 전체를 지배한다



자주 하는 실수: “분모가 0” 무시 — 그리고 지원 조건

분모 $b(a_t|s_t) = 0$ 시 정의 불가

실전 규칙

$b(a|s) = 0$ 인 (상태, 행동) 쌍은 **평가 대상에서 제외**하거나, 그 에피소드를 **통째로 버린다**. “분모가 0이면 그냥 1로 치자”는 **가장 흔한 실수**.

지원 조건 (support condition):

$$\text{supp}(\pi) \subseteq \text{supp}(b)$$

지원 조건: 목표 정책 ⊆ 행동 정책

on-policy vs off-policy: 트레이드오프

	On-policy (예: ε-greedy MC)	**Off-policy** (예: 중요도 샘플링)
데이터 수집 정책	학습하는 정책 **자체**	임의의 다른 정책 ($b \neq \pi$)
편향	없음 (평가 대상과 동일)	보정 전에는 있음, 보정 후에는 없음
분산	**낮음**	**높음** (ρ 폭증)
데이터 효율	낮음 (탐험 비용)	높음 (기존 데이터 재사용 가능)
구현 난이도	간단	ρ 계산 + 분산 다스림 필요
실용 예	5.2절 블랙잭 MC 제어	Week 9 DQN 경험재현, Week 11 PPO

off-policy 의 진짜 가치

"사람의 시연 데이터, 시뮬레이터에서 모은 데이터, 과거 버전의 정책이 모은 데이터"를 **재사용**할 수 있다는 점 — Week 12(모방학습)에서 사람의 동작 데이터를 강화학습에 어떻게 쓸지 다룰 때 이 off-policy 도구가 **필수 전제**.

자주 묻는 질문 (5.3) — 단일 샘플 vs 전체 궤적

"단일 샘플"과 "전체 궤적" 중요도 샘플링의 차이는?

- 위에서 쓰던 $\rho = \prod_t \frac{\pi(a_t|s_t)}{b(a_t|s_t)}$ 은 **전체 궤적 (full trajectory)** 중요도 — 에피소드 전체 확률의 비율.
- 상태가치 $V^\pi(s)$ 를 추정할 때는 **상태 s 이후 부분**(partial trajectory)만 쓰면 충분 — 분모가 "s 에서 시작하는 궤적 확률의 비율"이라 전체 궤적보다 **분산이 작다**.
- $Q^\pi(s,a)$ 를 추정할 때는 해당 (s,a) 이후 부분만 쓴다.

실전 원칙

가능한 한 짧은(목표와 관련된) 부분 궤적만 가중치로 쓴다 — 이것이 분산을 줄이는 가장 간단한 방법. (5.2절 블랙잭에서 "플레이어 합 12 이상 상태에서만 Q 를 학습"한 것도 같은 원리 — 시작 상태부터가 아니라 **관심 상태 이후**의 리턴만 쓴다.)

자주 묻는 질문 (5.3) — 뭐가 더 나은가

On-policy 와 off-policy 중 뭐가 더 나은가요?

On-policy

- 구현 **간단**, 분산 **작음**.
- 단, **학습하는 정책 자체로** 행동해야 하므로 탐험(무작위 행동)의 **비용**을 감수.

Off-policy

- 다른 정책(심지어 **사람의 시연 데이터**)이 모은 경험을 **재사용** $\Rightarrow$ 데이터 효율이 좋을 **수** 있다.
- 단, 중요도 샘플링의 **분산 문제**를 다뤄야.

Week 6 의 **Q-learning** 은 이 트레이드오프를 **다른 방식**(중요도 샘플링 없이 off-policy 를 구현)으로 풀어낸다.

"off-policy 의 데이터 효율이 항상 on-policy 보다 좋다"는 말은 **항상 참은 아니다** — ρ 의 분산이 극단적으로 커지는 경우(행동 정책과 목표 정책의 지원이 겹치는 정도가 낮음)에는 **오히려 나빠질 수 있다**.

MC 의 한계: 에피소드가 끝나야만 학습

MC 는 모델이 필요 없다는 강력한 장점이 있지만, **에피소드가 끝나야만** 학습할 수 있다는 근본적 제약이 있다 — 리턴 G_t 자체가 **에피소드 끝까지의 보상을 다 더한 값**이기 때문.

continuing task(끝이 없는 과업)라면 MC 는 아예 쓸 수 없다

ending 이 없으면 "에피소드 끝까지의 보상 합"이라는 G_t 자체가 **정의되지 않는다**.

대신 **H 스텝 절단 리턴**

$$G_t^{(H)} = R_t + \gamma R_{t+1} + \cdots + \gamma^{H-1} R_{t+H-1}$$

로 대체할 수 있지만 **H 를 몇으로 잡을지**를 사람이 정해야 한다 — H 가 너무 **짧으면 편향**(먼 미래 보상 무시), 너무 **길면 분산**(먼 미래 무작위성 누적).

매 스텝 $+1$, $\gamma = 0.9$ 인 continuing task — **진짜 리턴** $\frac{1}{1-\gamma} = 10$:

H	절단 리턴 $G_t^{(H)}$
5	4.0951
20	8.7842
50	9.9485
200	10.0000

"에피소드가 끝나면 **알아서 결정되는** 문제"가 "H 를 **수동으로 고르는** 문제"로 변했다.

Week 6 의 시간차 학습

"에피소드가 끝나기를 기다리지 않고, **한 스텝만 보고도 즉시 갱신한다**"는 완전히 다른 절충안으로 이 문제를 푼다.

실습: 중요도 샘플링 직접 구현

```
GAMMA = 0.9

def sample_episode(rng, start=2, n_max=200):
    """행동 정책 b(50-50)로 한에피소드샘플링 ."""
    s, ep = start, []
    for _ in range(n_max):
        a = 0 if rng.random() < 0.5 else 1
        ns, r, done = step(s, a)
        ep.append((s, a, r)); s = ns
        if done: break
    return ep

def estimate(rng, n_episodes, start=2):
    """세 가지추정치 : (1) 그냥평균 (2) 중요도가중평균 ."""
    naive, isw = [], []
    for _ in range(n_episodes):
        ep = sample_episode(rng, start)
        G = 0.0
        for t in reversed(range(len(ep))):
            G = ep[t][2] + GAMMA * G
        naive.append(G)                          # 편향됨: V^b(start)
        all_right = all(a == 1 for _, a, _ in ep)
        L = len(ep)
        rho = (2.0 ** L) if all_right else 0.0   # b=50-50, pi=right
        isw.append(rho * G)                      # 편향없음 : V^pi
    return np.mean(naive), np.std(naive), np.mean(isw), np.std(isw)
```

같은 데이터, 다른 가중치 ⇒ **다른 정책의 가치를 추정** — 중요도 샘플링의 본질.

실습 결과: 편향 vs 분산이 눈에 보인다

N	naive mean	naive sd	IS mean	IS sd
10	-0.440	0.703	$+0.720$	1.440
100	-0.011	0.776	$+0.900$	1.559
1000	$+0.033$	0.767	$+0.878$	1.546
10000	$+0.002$	0.775	$+0.887$	1.551
20000	$+0.004$	0.774	$+0.892$	1.554

- naive 평균: $N = 10$ 에 -0.440 이라 $N = 20000$ 에 0.004 로 **0에** 수렴 (진짜값 $V^b = 0$ — 편향 없이 "다른 정책의 가치를 잘 추정").
- IS 평균: $N = 10$ 에 $+0.720$ 이라 $N = 20000$ 에 $+0.892$ 로 **0.9에** 수렴 (진짜값 $V^\pi = 0.9$ — 편향 없이 **목표 정책**의 가치를 잘 추정).

IS 쪽의 표준편차(≈ 1.55)가 평균(≈ 0.9)의 거의 2배인 것이 **"분산 폭증"의 직접적 증거**.

추가 확인: ρ 분포가 길이에 따라 퍼진다

L	$\mathsf{P}(\rho > 0) = 2^{-L}$	/	$\mathsf{Var}[\rho] = 2^L - 1$
$L = 1$	0.50000	/	1
$L = 2$	0.25000	/	3
$L = 3$	0.12500	/	7
$L = 4$	0.06250	/	15
$L = 6$	0.01562	/	63
$L = 8$	0.00391	/	255
$L = 10$	0.00098	/	1023

$$P(\rho = 4) = 0.250, \qquad P(\rho = 0) = 0.750$$

$\mathbb{E}[\rho] = 1.001$ (이론 1), $\quad \mathsf{Var}[\rho] = 3.002$ (이론 $2^2 - 1 = 3$)

L 이 **커질수록** $\mathsf{Var}[\rho] = 2^L - 1$ (지수적 폭증).

자주 하는 실수: ρ 의 분산을 “평균 에피소드 길이”에 대입하기

평균 길이에 대입하는 것은 잘못이다

$$\mathbb{E}[2^L] \;\geq\; 2^{\mathbb{E}[L]}$$

의미

이것이 “에피소드 길이가 불규칙한 환경에서 off-policy 가 **특히 어렵다**”는 말의 정확한 의미.

몬테카를로의 한 줄 요약

“모델을 몰라도, 충분히 많이 시도해보면 평균이 진실에 수렴한다”

— 통계학의 가장 기본적인 원리를 강화학습에 그대로 옮긴 것.

다만 그 대가로, 결과를 알기 위해 항상 게임이 끝날 때까지 기다려야 한다.

Week 6 예고: 시간차 학습 (Temporal-Difference)

“기다림 없이 배운다” — 에피소드가 끝나기를 기다리지 않고 **한 스텝만 보고도 즉시 갱신**. 이번 장의 “에피소드를 끝까지 기다린다”는 대가를 없애는 것이 바로 TD.

이번 주차 정리

1. **MC 예측 (5.1)** — **기댓값 = 샘플 평균**이라는 통계학 원리를 강화학습에 그대로 이식. 잔디깎는 기 MDP 에서 전이확률 단 한 줄도 쓰지 않은 채 DP 의 정확해와 **같은 숫자**(0.286 등)에 도달. **첫방문** MC 가 무편향인 이유는 **마르코프 성질 + 대수의 법칙** 두 줄. 뒤에서부터 훑으면 "마지막 방문"을 세게 되어 **편향된 추정치**(0.756 vs 0.713)가 됨 — 코딩 실수가 통계적 편향으로 바뀌는 대표적 예.
2. **MC 제어 (5.2)** — V 대신 $Q(s,a)$ 를 추정하고 arg max 로 결정론적 정책을 뽑음. ε-greedy 로 블랙잭 30~40만 에피소드를 돌리면 **교과서 기본 전략표**(17 이상 스틱, 12~13 히트)가 학습만으로 재현(104/110 → 105/110 일치). 학습 대상은 **행동 정책** π **의** Q^{π} (on-policy) — Q^{*} 보다 항상 약간 낮게 측정. "방문 횟수 = 신뢰도".
3. **중요도 샘플링 (5.3)** — off-policy 의 편향을 확률의 비율 $\rho = \prod_t \frac{\pi(a_t|s_t)}{b(a_t|s_t)}$ 로 보정. 5칸 회랑에서 손으로 확인: 편향은 없애주되 $\mathrm{Var}[\rho] = 2^L - 1$ 로 **지수적으로 분산 폭증**. "분모 0 무시" 금지(지원 조건 $\mathrm{supp}(\pi) \subseteq \mathrm{supp}(b)$), on/off-policy 트레이드오프를 정리.

다음 주차 예고: Week 6. 시간차 학습 (TD)

- TD는 한 스텝만 보고 즉시 갱신
- SARSA와 Q-learning 비교